



Data Everywhere: Using and Sharing Scientific Data with Pelican

Andrew Owen & Christina Koch

Research Computing Facilitators

Center for High Throughput Computing

University of Wisconsin - Madison

Brian Bockelman

Investigator, Pelican Platform PI

Morgridge Institute for Research

University of Wisconsin - Madison



Setup

All instructions, materials are in Github repository

github.com/pelicanplatform/training-client



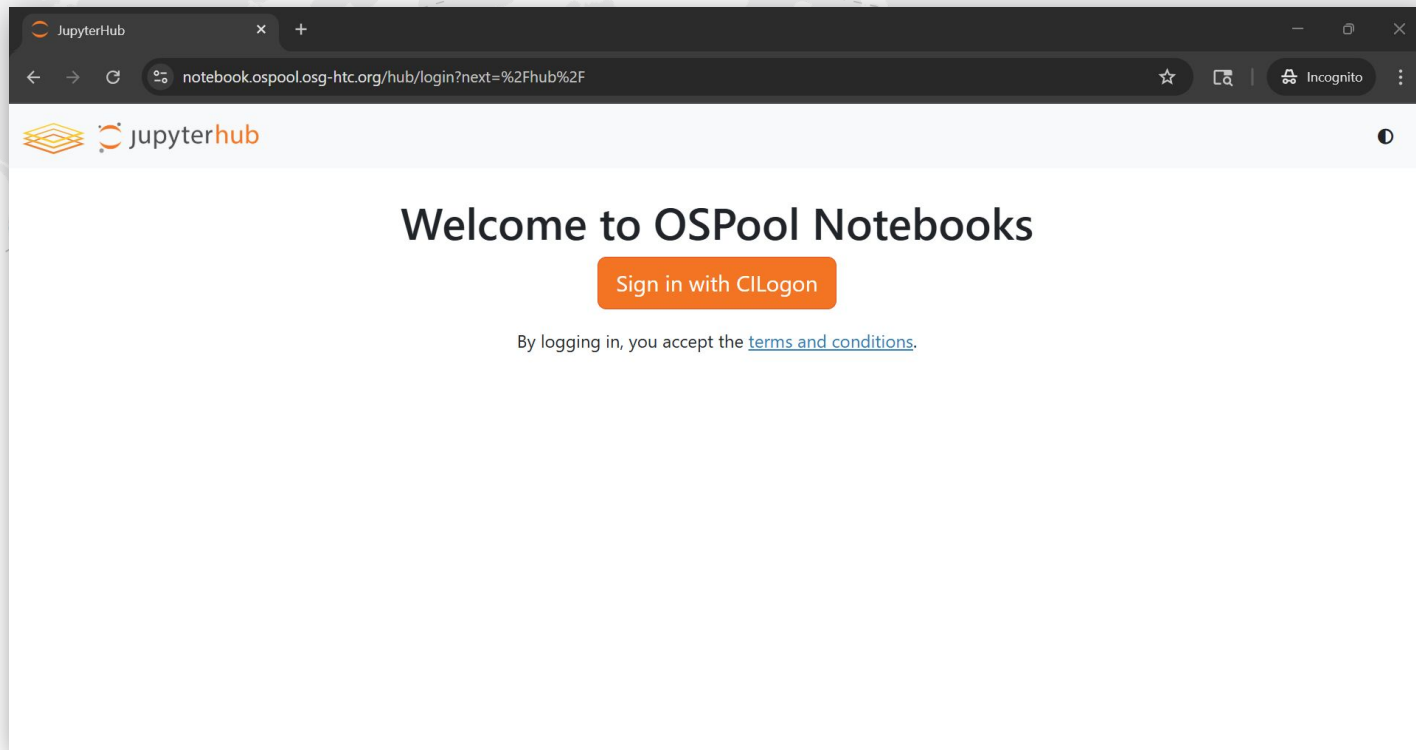
Quickstart

1. Sign in to a Guest notebook at notebook.ospool.osg-htc.org using your institution ID.
2. Open a "Terminal" tab in the Jupyter Notebook



Quickstart

notebook.ospool.osg-htc.org





Quickstart

notebook.ospool.osg-htc.org

Welcome To The CILogon Open

cilogon.org/authorize?response_type=code&redirect_uri=https%3A%2F%2Fnotebook.ospool.osg-htc.org%2Fhub%2Foauth_callback&clien...

CILogon

Consent to Attribute Release

OSG's OSPool Notebooks requests access to the following information. If you do not approve this request, do not proceed.

- Your CILogon user identifier
- Your name
- Your email address
- Your username and affiliation from your identity provider

Select an Identity Provider

University of Wisconsin-Madison

☐ Remember this selection

Log On

Choose **your** institution!



Quickstart

notebook.ospool.osg-htc.org

JupyterHub

notebook.ospool.osg-htc.org/hub/spawn

jupyterhub Home Token

aowen4@wisc.edu Logout

Server Options

- ☒ **Guest Account: Explore OSPool Notebooks**
Explore OSPool notebooks via two tutorials: one for analyzing many independent files (tutorial-fastqc), and one for running many random predictive simulations (tutorial-ScalingUp-R).
- ☐ **AP40 Account: Full OSPool Access**
Submit HTCondor jobs to the OSPool using your AP40 account. Perform data analysis using libraries from the Julia, Python, and R communities.

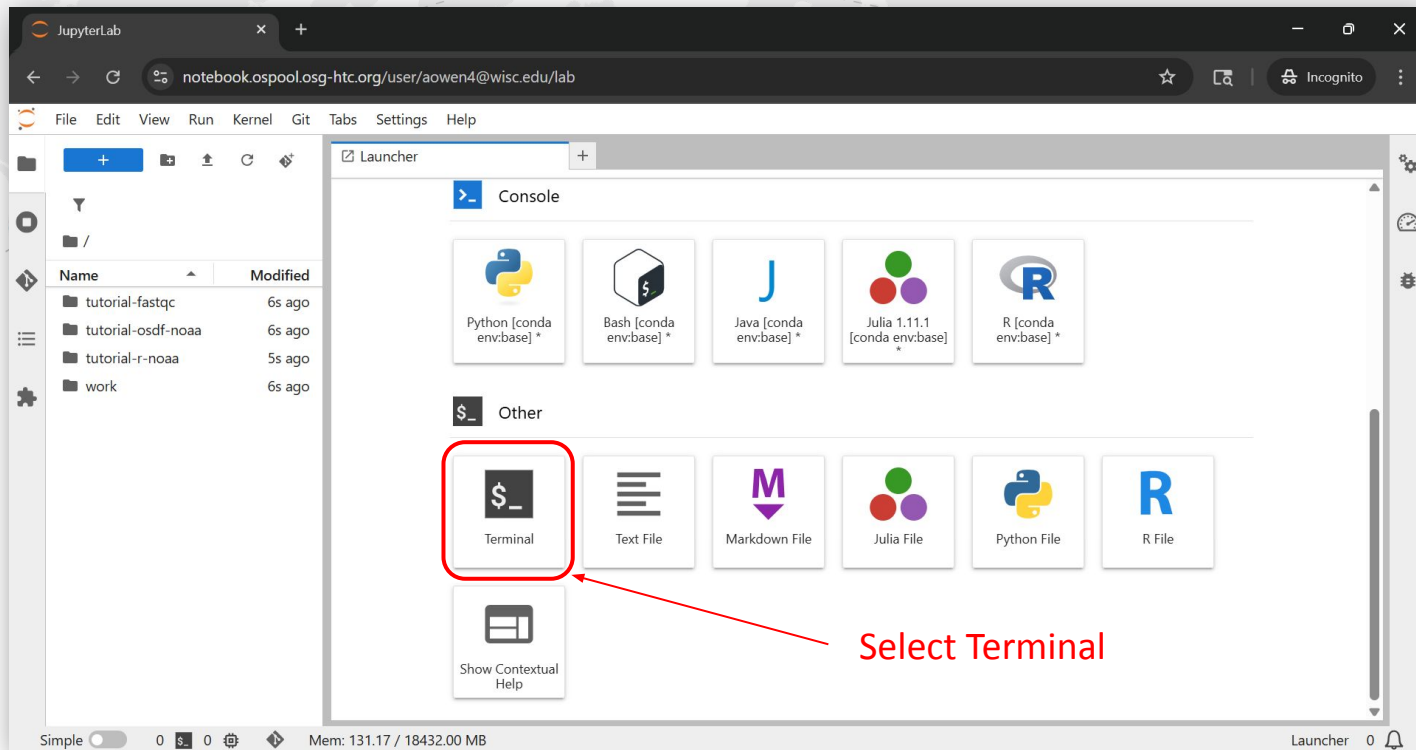
Start

Select
Guest



Quickstart

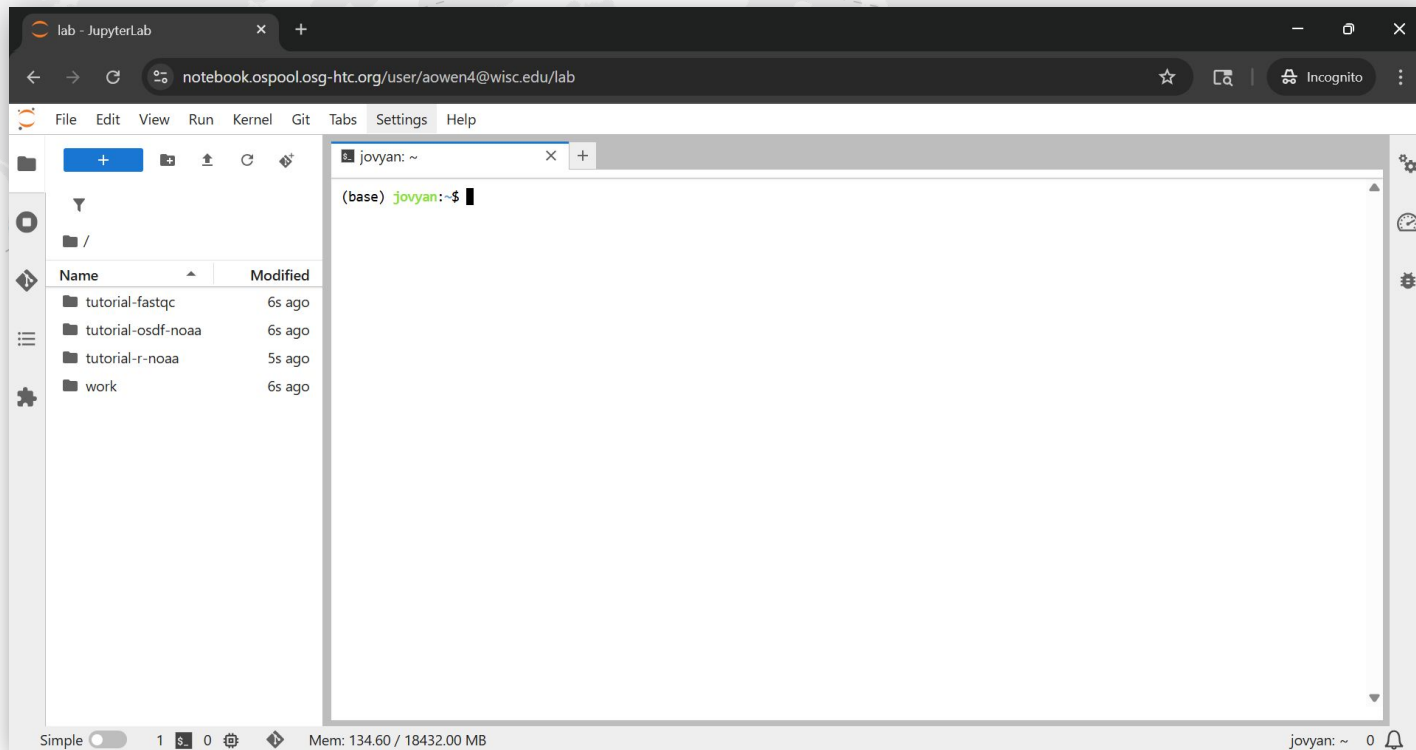
notebook.ospool.osg-htc.org





Quickstart

notebook.ospool.osg-htc.org





Quickstart

1. Sign in to a Guest notebook at notebook.ospool.osg-htc.org using your institution ID.
2. Open a "Terminal" tab in the Jupyter Notebook
3. In a separate window, go to osg-htc.org/services/osdf/data and choose a repository from the table
4. Click on the repository row and copy the command under "Download a Public Object"
5. Paste and the enter command into "Terminal" tab of Jupyter Notebook



Quickstart

osg-htc.org/services/osdf/data

The screenshot shows a web browser window with the URL `osg-htc.org/services/osdf/data`. The page has a yellow header with navigation links: Services, Campuses, Community, Contact, About, and Docs. On the right of the header are icons for user profile, phone, PATH, and a circular logo. The main content area has the heading "Explore the OSDF" followed by the text "The OSDF service connects the nation's scientific data repositories to computing infrastructure." Below this is the heading "Repositories available via the OSDF" and the text "46 repositories are connected to the OSDF to help deliver scientific data. The table below illustrates the datasets accessible via the OSDF." There is a search input field with the placeholder "Type a keyword...". Below the search field is a table with three visible columns: "Name", "Organization", and "Field Of Science". Each column has a dropdown arrow. To the right of the table is a red button with a question mark icon and the text "Contact Us!".

lab - JupyterLab

Explore the OSDF | OSG

osg-htc.org/services/osdf/data

Services ▾ Campuses ▾ Community ▾ Contact About ▾ Docs ▾

PATH

Explore the OSDF

The OSDF service connects the nation's scientific data repositories to computing infrastructure.

Repositories available via the OSDF

46 repositories are connected to the OSDF to help deliver scientific data. The table below illustrates the datasets accessible via the OSDF.

Type a keyword...

Name ▾	Organization ▾	Field Of Science ▾
--------	----------------	--------------------

Contact Us!



Quickstart

osg-htc.org/services/osdf/data

Amazon Web Services Open Data	Amazon Web Services, Inc.	Multidisciplinary Studies.	Dataset Catalog
Gravitational Wave Open Science Center	California Institute of Technology	Astronomy and Astrophysics	Learn More
SPIN4D Data Release 1	University of Hawaii-Moana	Physical Sciences	View Datasets
NCAR Research Data Archive	NSF National Center for Atmospheric Research	Physical Sciences	Dataset Catalog
RouteViews	University of Oregon	Computer Systems Networking and Telecommunications	View Datasets
BurnPro3D	University of California, San Diego	Natural Resources and Conservation	Dataset Catalog
NCEI Water Column Sonar Data	National Oceanic and Atmospheric Administration	Fishing and Fisheries Sciences and Management	View Datasets Contact Us!

Click on
a row



Quickstart

osg-htc.org/services/osdf/data

Download a Public Object

With Pelican Client on the Command Line

```
pelican object get osdf:///routeviews/chicago/route-views.chicago/bgpdata/2025.03/RIBS/rib.20250319
```

From Your Browser

[Click to Download Public Object](#)

1 Dataset(s)
available via the OSDF

114 Terabytes
of data connected

[Contact Us!](#)

Copy the
command



Quickstart

notebook.ospool.osg-htc.org

lab - JupyterLab

Explore the OSDF | OSG

notebook.ospool.osg-htc.org/user/aowen4@wisc.edu/lab

File Edit View Run Kernel Git Tabs Settings Help

+

File browser:

Name	Modified
tutorial-fastqc	6s ago
tutorial-osdf-noaa	6s ago
tutorial-r-noaa	5s ago
work	6s ago
rib.20250319.0400...	now

Terminal:

```
(base) jovyan:~$ pelican object get osdf:///routeviews/chicago/route-views.chicago/bgpdata/2025.03/RIBS/rib.20250319.0400.bz2 ./rib.20250319.0400.bz2 22.00 MiB / 72.45 MiB [=====] 0s ] 0.00 b/s
```

Simple 1 \$ 0 Mem: 191.07 / 18432.00 MB

jovyan: ~ 0

Paste and run the command!



Quickstart

notebook.ospool.osg-htc.org

The screenshot shows a JupyterLab interface with a terminal window. The terminal displays the following commands and output:

```
(base) jovyan:~$ pelican object get osdf:///routeviews/chicago/route-views.chicago/bgpdata/2025.03/RIBS/rib.20250319.0400.bz2 ./
(base) jovyan:~$ ls -lh
total 73M
-rw-r--r-- 1 jovyan users 73M May 27 22:05 rib.20250319.0400.bz2
drwxr-sr-x 7 jovyan users 184 May 27 21:58 tutorial-fastqc
drwxr-sr-x 6 jovyan users 4.0K May 27 21:58 tutorial-osdf-noaa
drwxr-sr-x 8 jovyan users 4.0K May 27 21:58 tutorial-r-noaa
drwsrwsr-x 1 jovyan users 60 May 27 21:58 work
(base) jovyan:~$
```

The left sidebar shows a file browser with the following files and their modification times:

Name	Modified
tutorial-fastqc	6s ago
tutorial-osdf-noaa	6s ago
tutorial-r-noaa	5s ago
work	6s ago
rib.20250319.0400...	5s ago

Below the terminal window, the text "Done!" is displayed with a party popper icon.

***Keep the notebook open for later
hands-on exercises!***



What did you just do? (Introduction to the Pelican Platform)



What did you do?

There are three main things that you did:

1. Found a data repository available via the OSDF
2. Found the Pelican URL for that data repository
3. Downloaded the corresponding object using Pelican

Let's examine these steps to get an introduction to Pelican



Exploring the OSDF

The Open Science Data Federation (OSDF) connects disparate dataset repositories into a **single, nationwide data distribution network**.

Leveraging the OSDF, **providers can make their datasets available to a wide variety of compute users**, from browsers to Jupyter notebooks to high throughput computing environments.

The OSDF is part of the OSG Fabric of Services, **running software developed by the Pelican Platform**.

<https://osg-htc.org/services/osdf>



Exploring the OSDF

osg-htc.org/services/osdf/data

46

Repositories

available via the OSDF

127

Objects per Second

transferred on average

129

Petabytes

of data delivered (12 mos)

... and growing!



Exploring the OSDF

The OSDF is powered by the **Pelican Platform**, a software suite for creating **data federations**, which serves to unite **data contributors** with **data consumers** under a single namespace and access model.

- **Data contributors** can connect existing datastores to a **data federation** while maintaining their access policies
- **Data consumers** can easily access data objects in the **data federation** without needing to know anything about the underlying infrastructure



Exploring the OSDF

osg-htc.org/services/osdf/data

Repositories available via the OSDF

46 repositories are connected to the OSDF to help deliver scientific data. The table below illustrates the datasets accessible via the OSDF.

Name	Organization	Field Of Science	
Amazon Web Services Open Data 	Amazon Web Services, Inc.	Multi/Interdisciplinary Studies.	Dataset Catalog
Gravitational Wave Open Science Center 	California Institute of Technology	Astronomy and Astrophysics	Learn More
SPIN4D Data Release 1 	University of Hawaii-Moana	Physical Sciences	View Dataset  Contact Us!
NSAID Data Release 1	NSAID Data Release 1	Physical Sciences	View Dataset



Exploring the OSDF

osg-htc.org/services/osdf/data

Repositories available via the OSDF

46 repositories are connected to the OSDF to help deliver scientific data. The table below illustrates the datasets accessible via the OSDF.

Name	Organization	Field Of Science
------	--------------	------------------



Your repository could be in this list!

Please reach out if you are even a tiny bit interested:

support@osg-htc.org



Exploring the OSDF

osg-htc.org/services/osdf/data

Every repository gets a unique "Pelican URL"

Data consumers only need the Pelican URL to access the data!

Download a Public Object

With Pelican Client on the Command Line

```
pelican object get osdf:///routeviews/chicago/route-views.chicago/bgpdata/2025.03/RIBS/rib.20250319
```



Anatomy of a Pelican URL

The tale of Pelican in 4 parts:

- The "**protocol**"
- The "**discovery URL**"
- The "**namespace prefix**" (aka "**namespace**")
- The "**object name**"

For example:

pelican://**osg-htc.org**/**pelicanplatform**/**test**/**hello-world.txt**



Anatomy of a Pelican URL

The "**protocol**"

- Open source web protocol **pelican**
- Extension of HTTP
- Anyone can use it to interact with a Pelican Data Federation!

pelican://_____



Anatomy of a Pelican URL

The "**discovery URL**"

- The **pelican** protocol is used to interact with a Pelican Federation
- The **discovery URL** tells the protocol which Federation to talk to!

The **discovery URL** for the OSDF is **osg-htc.org**

pelican://osg-htc.org/_____



Anatomy of a Pelican URL

The OSDF "protocol"

- Pelican provides a shortcut "protocol" for specifying the OSDF!

osdf:///_____

is exactly equivalent to

pelican://osg-htc.org/_____



Anatomy of a Pelican URL

The "discovery URL"

- The **pelican** protocol is used to interact with a Pelican Federation
- The **discovery URL** tells the protocol which Federation to talk to!

The **discovery URL** for the OSDF is **osg-htc.org**

pelican://osg-htc.org/_____

osdf:///_____



Anatomy of a Pelican URL

The "**namespace prefix**" (aka "**namespace**")

- When a **data provider** connects their repository to a Federation, they make it available under a particular **namespace**
- The **data provider** gets to decide who gets to access their data and has control over all Pelican URLs that start with the same prefix!

The Pelican Platform owns the **pelicanplatform** namespace in the OSDF

pelican://**osg-htc.org**/**pelicanplatform**/-----

osdf:///b**pelicanplatform**/-----



Anatomy of a Pelican URL

The "object name"

- Corresponds to data in the repository accessible via the **namespace** connected to the Federation
- The process of connecting the **namespace** to the Federation includes mapping the storage into individual object names

pelican://osg-htc.org/pelicanplatform/test/hello-world.txt

osdf:///pelicanplatform/test/hello-world.txt



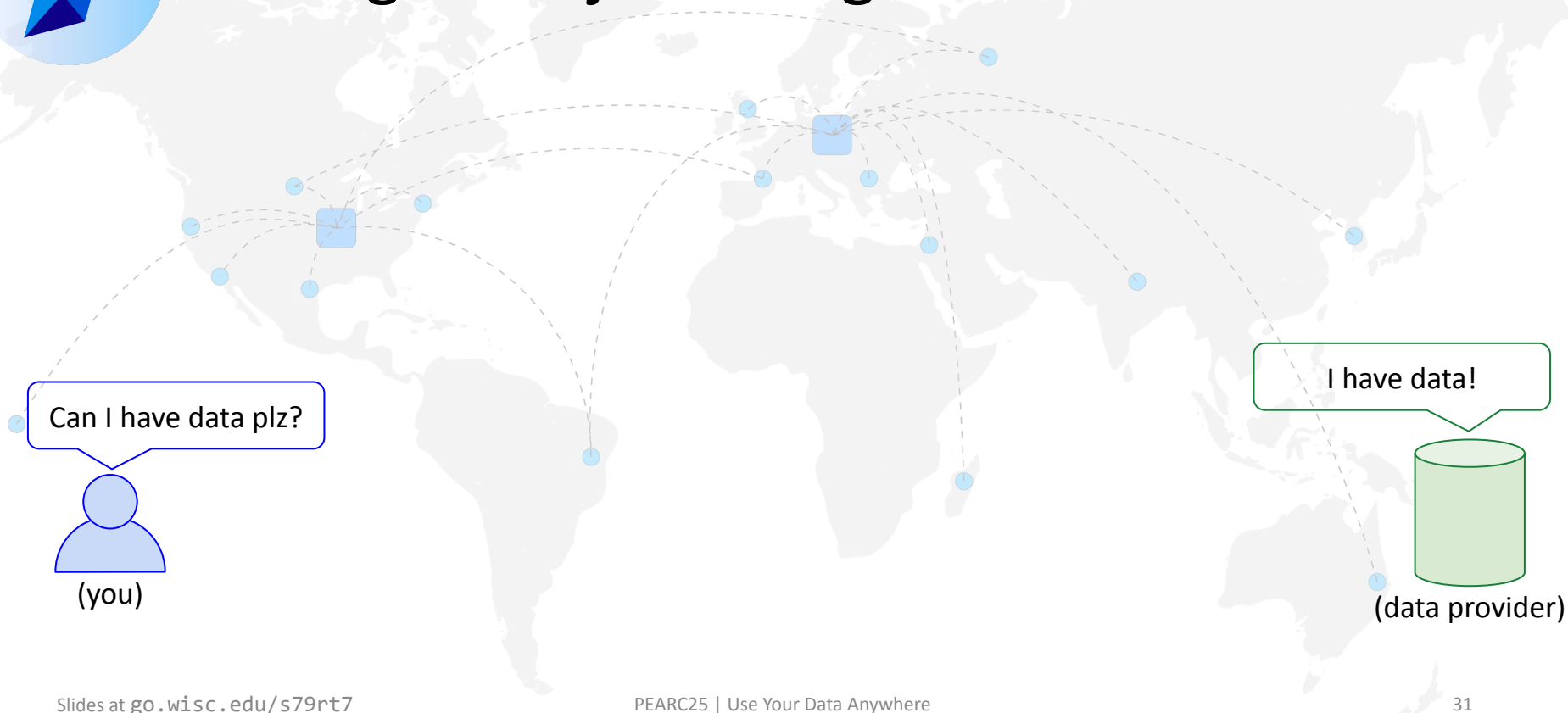
Getting an object using a Pelican URL

If you know the Pelican URL for an object, you can download (get) it!

But how?

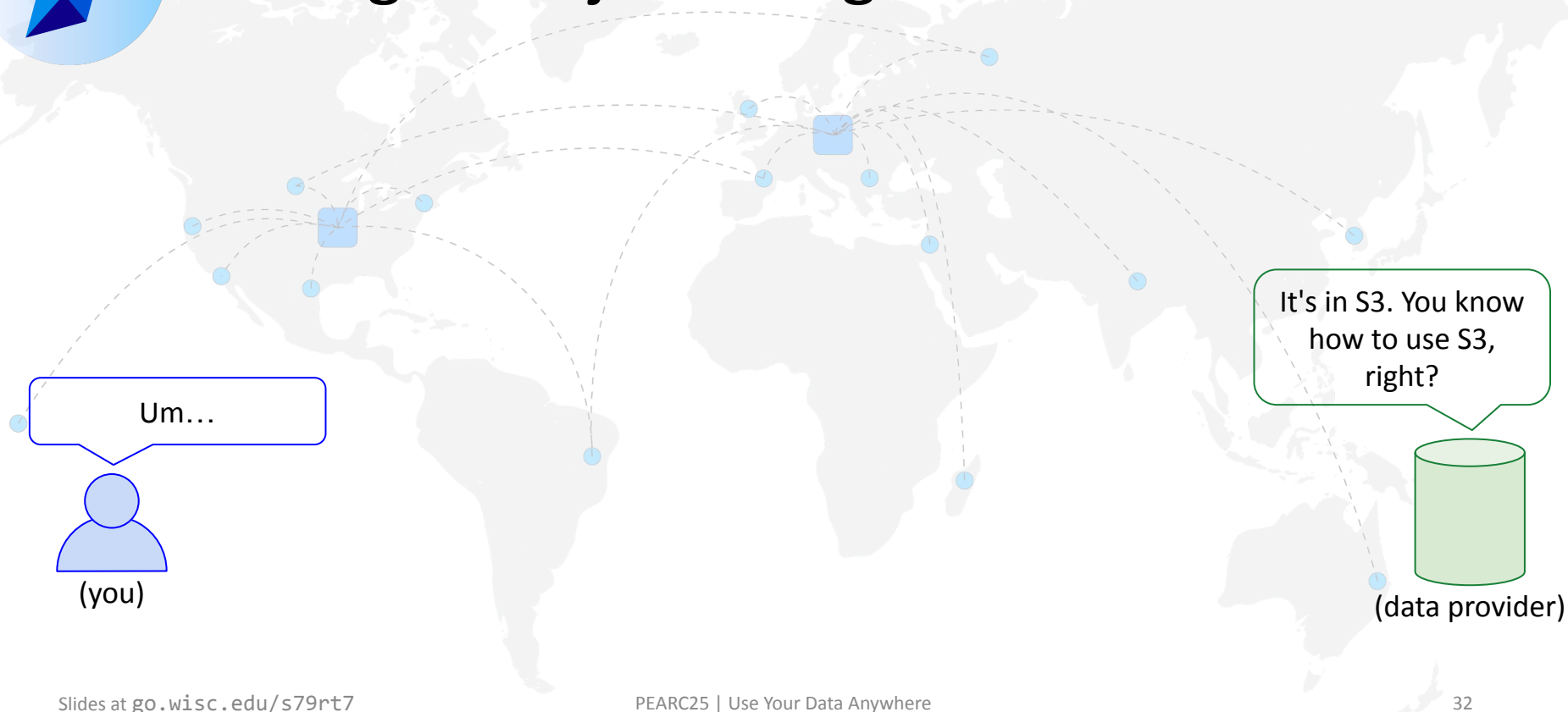


Getting an object using a Pelican URL





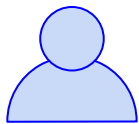
Getting an object using a Pelican URL





Getting an object using a Pelican URL

pelican://osg-htc.org/



(you)



(data provider)

Data Federation



Getting an object using a Pelican URL

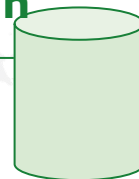
`pelican://osg-htc.org/`



(you)



Origin



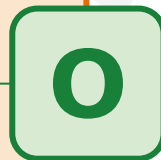
(data provider)

Data Federation

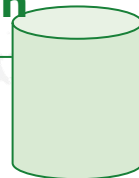


Getting an object using a Pelican URL

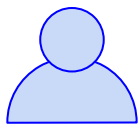
`pelican://osg-htc.org/namespace/`



Origin



(data provider)



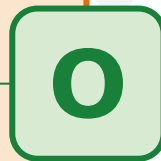
(you)

Data Federation



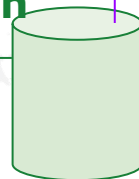
Getting an object using a Pelican URL

`pelican://osg-htc.org/namespace/object`



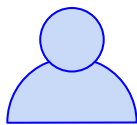
Origin

object



(data provider)

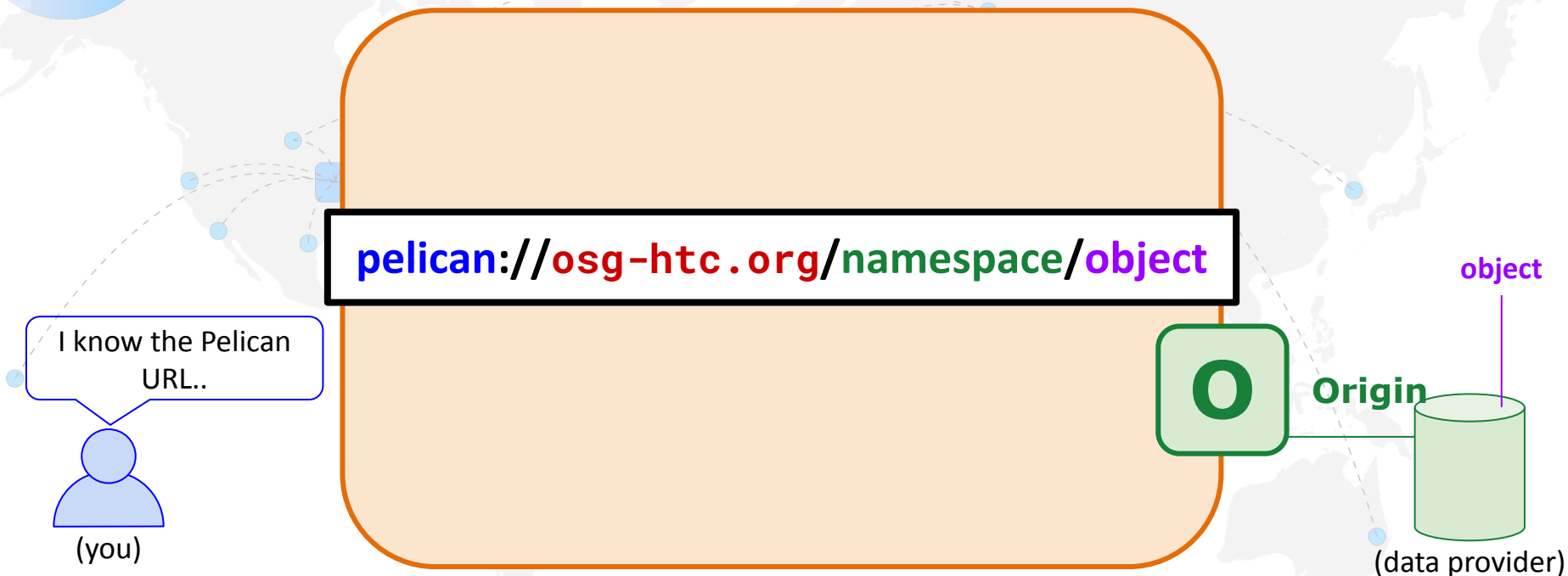
Data Federation



(you)



Getting an object using a Pelican URL



Data Federation



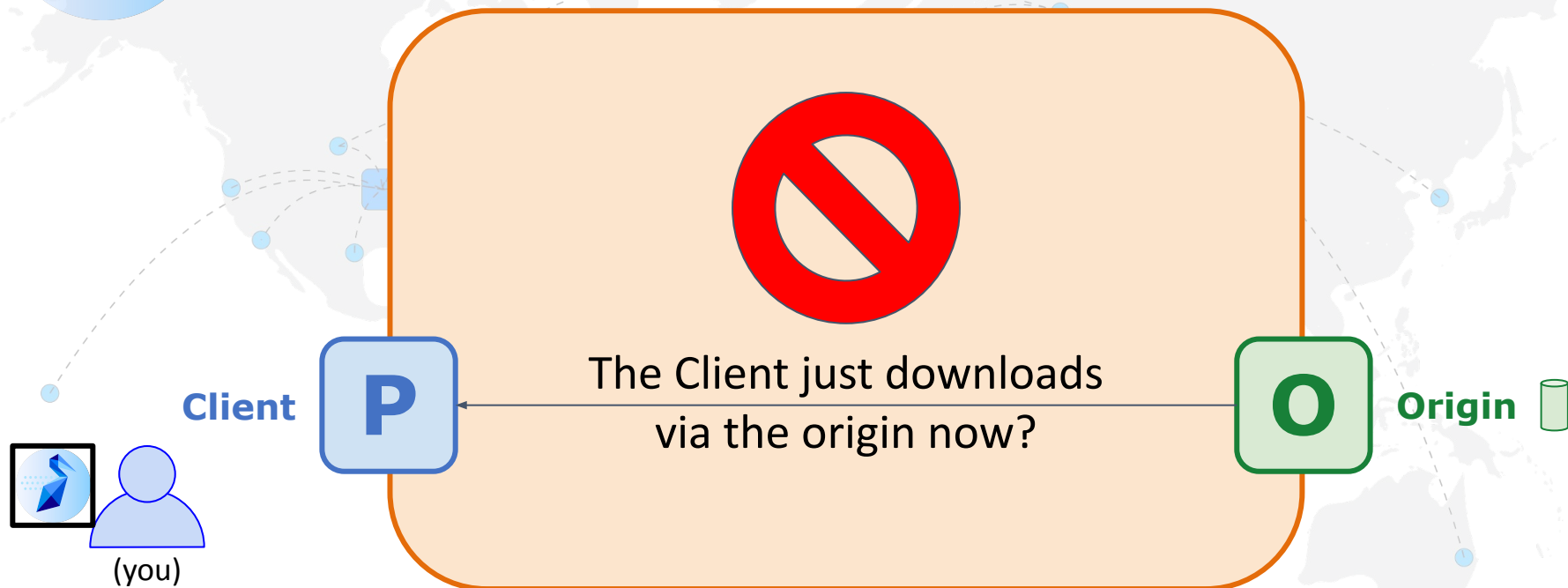
Getting an object using a Pelican URL



Data Federation



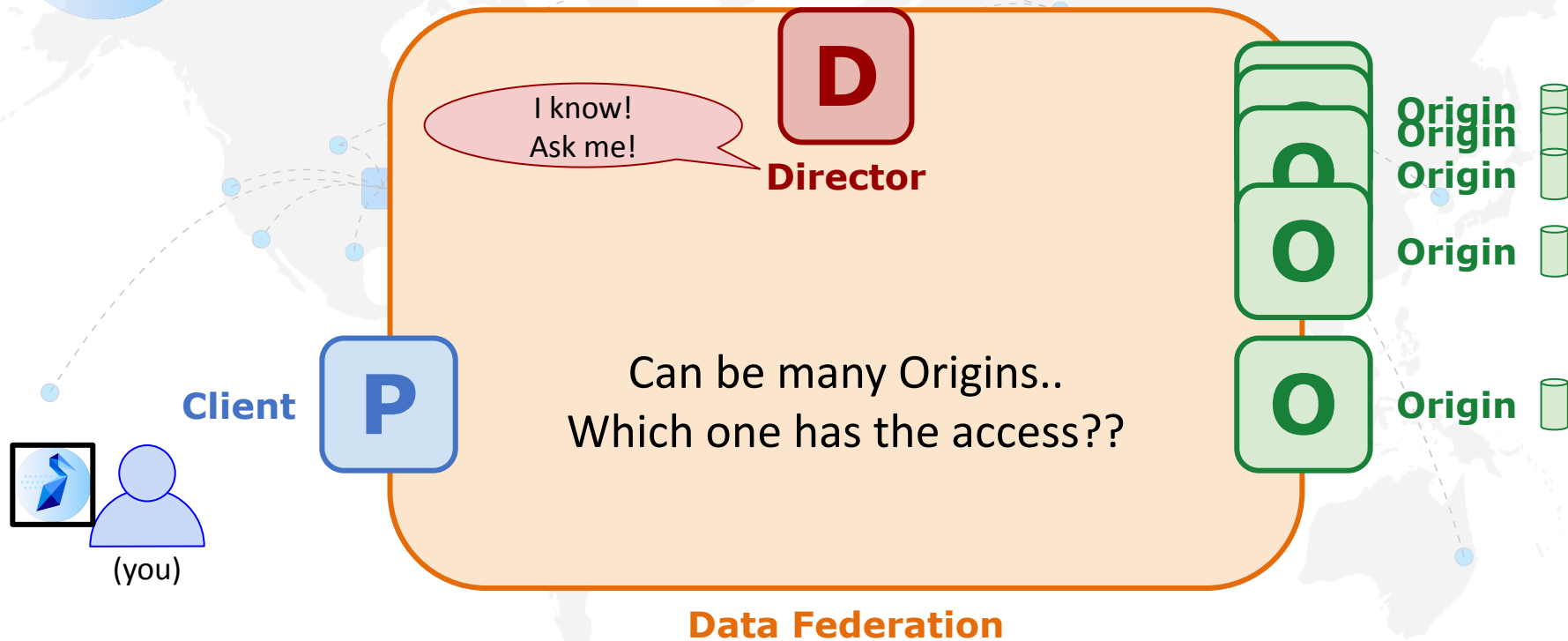
Getting an object using a Pelican URL



Data Federation



Getting an object using a Pelican URL



Data Federation

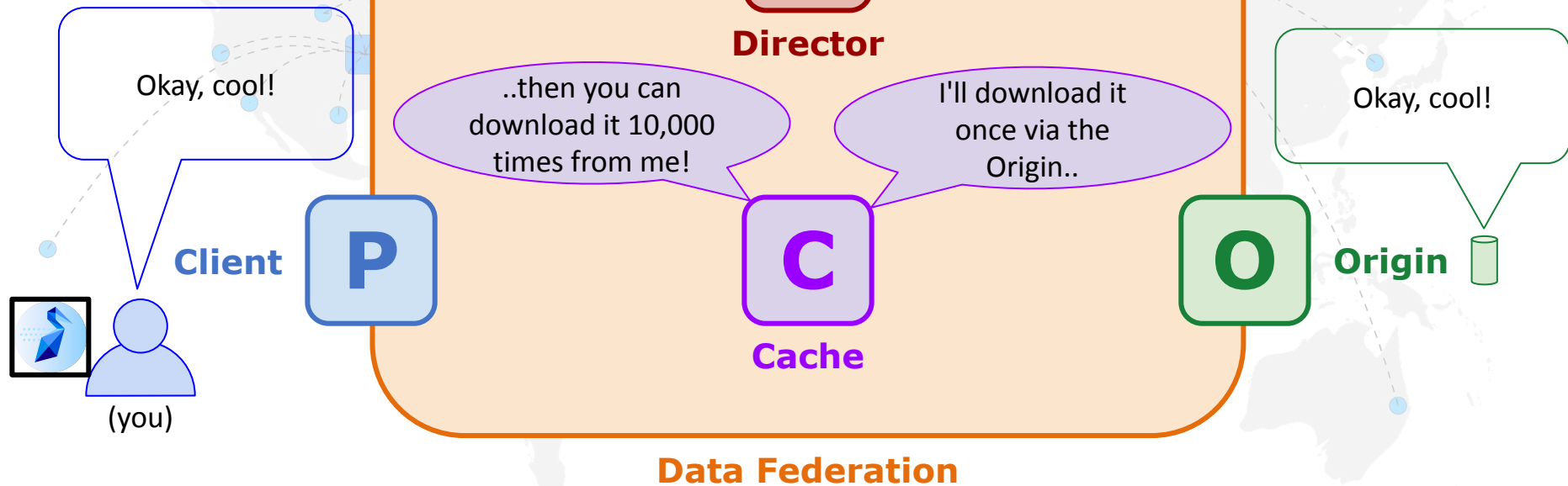


Getting an object using a Pelican URL



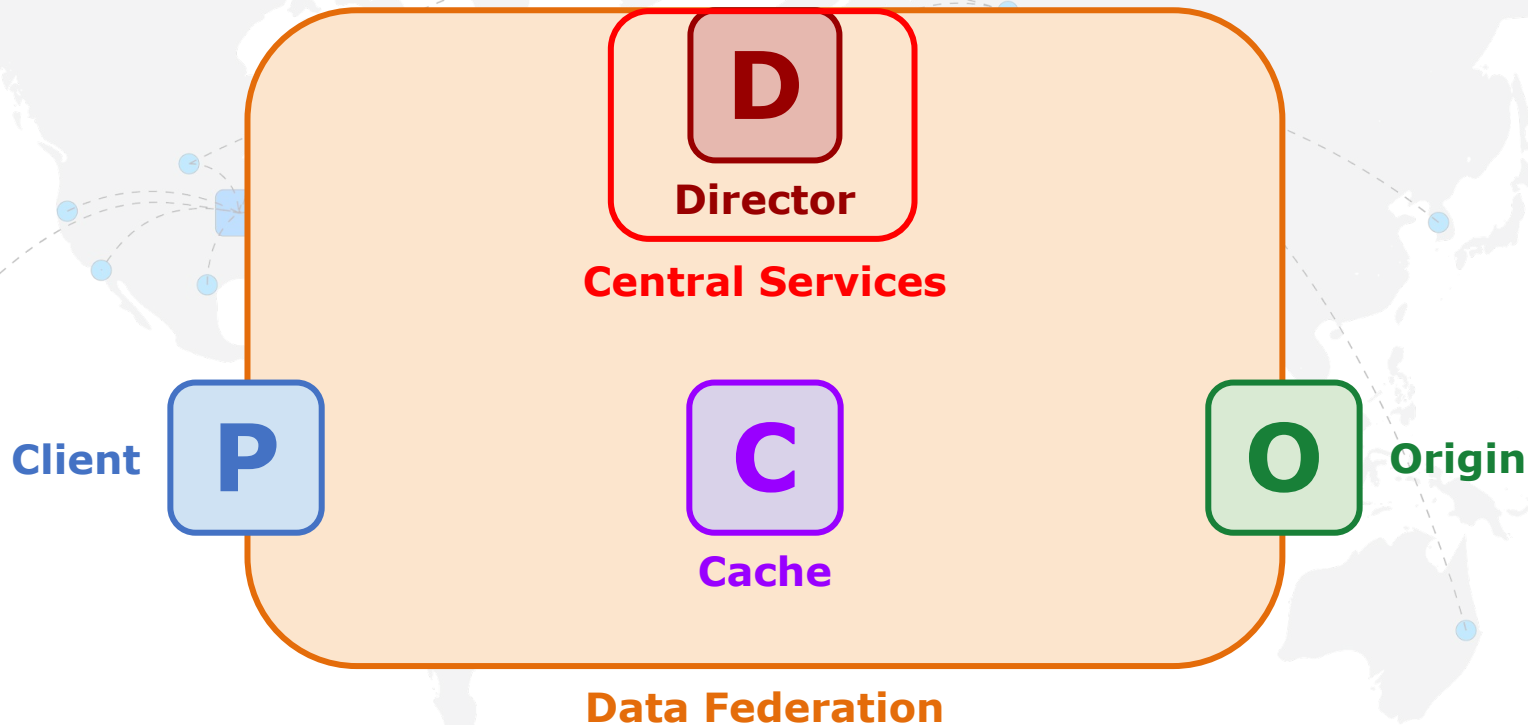


Getting an object using a Pelican URL



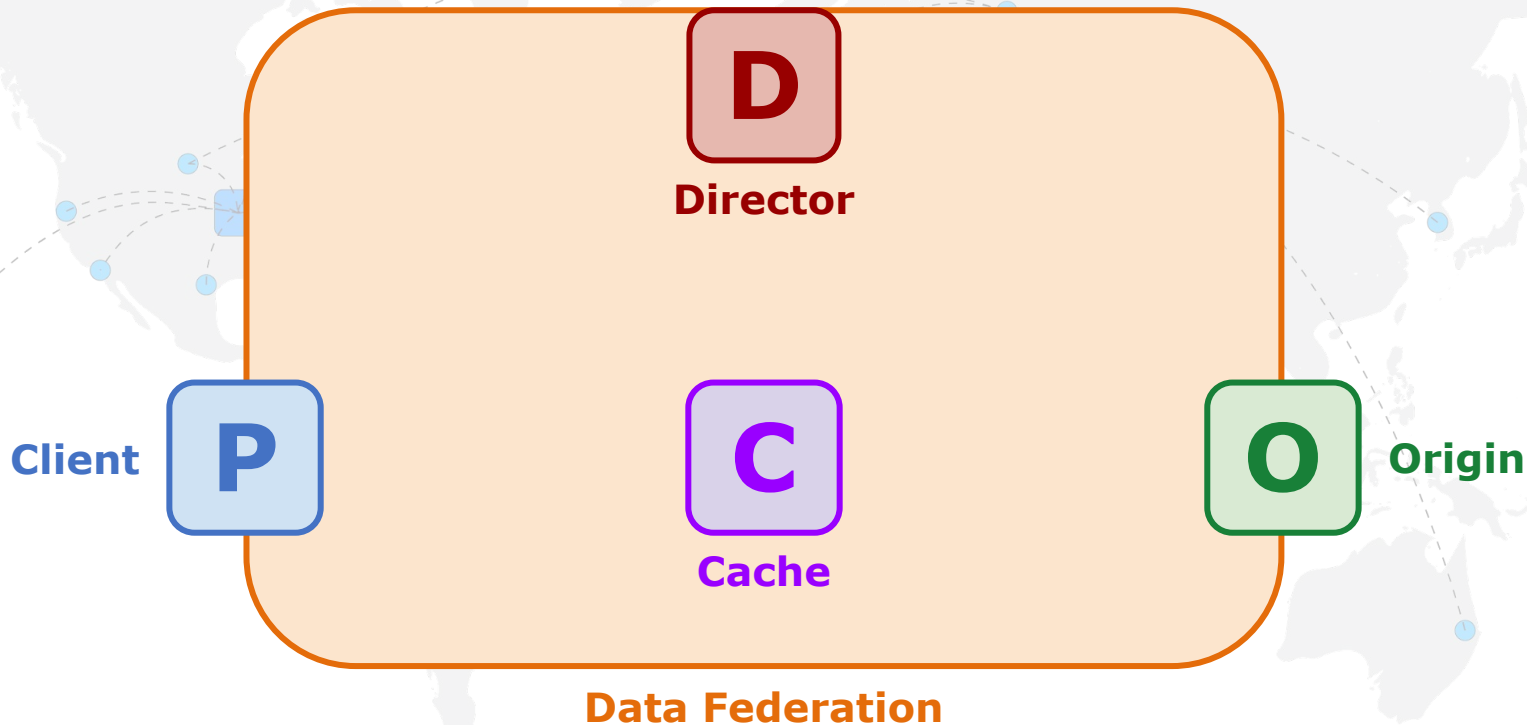


Getting an object using a Pelican URL



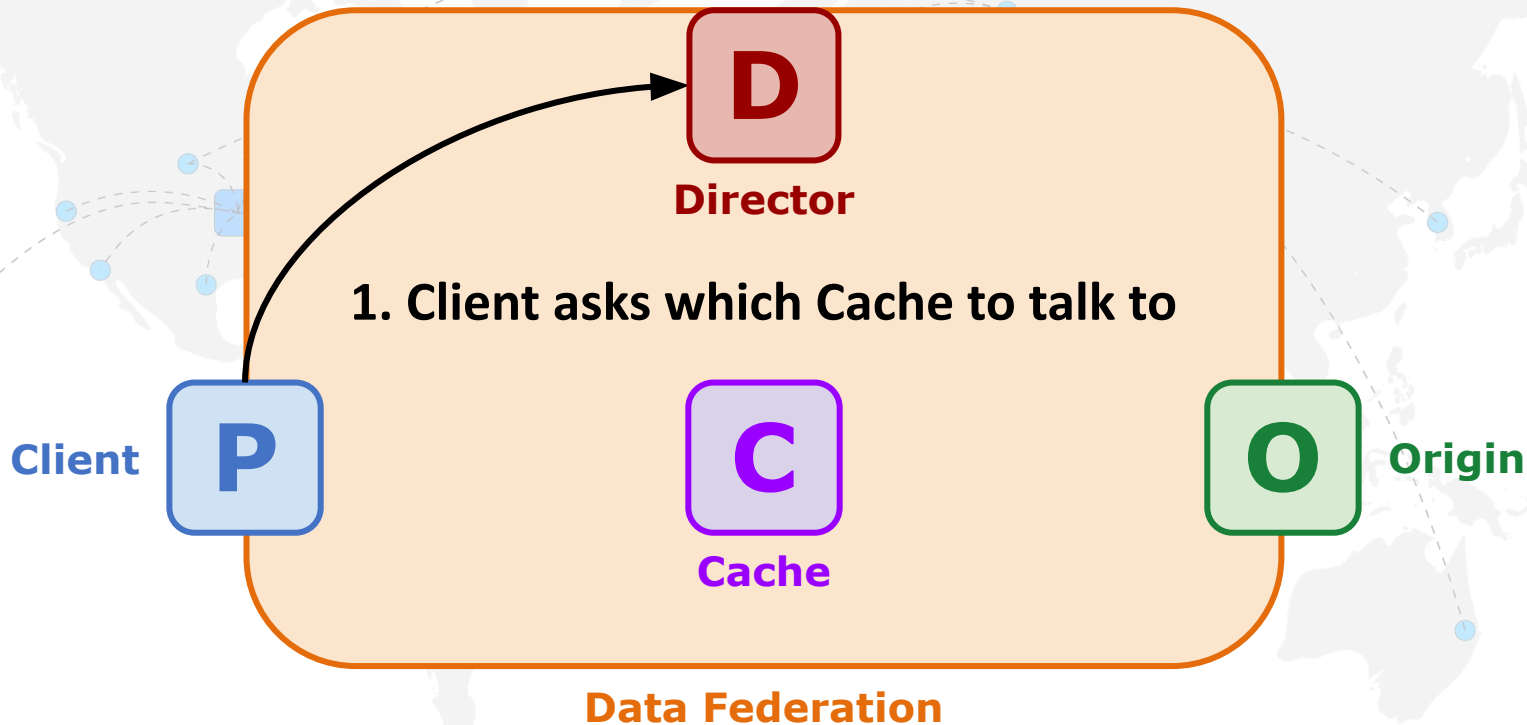


Getting an object using a Pelican URL



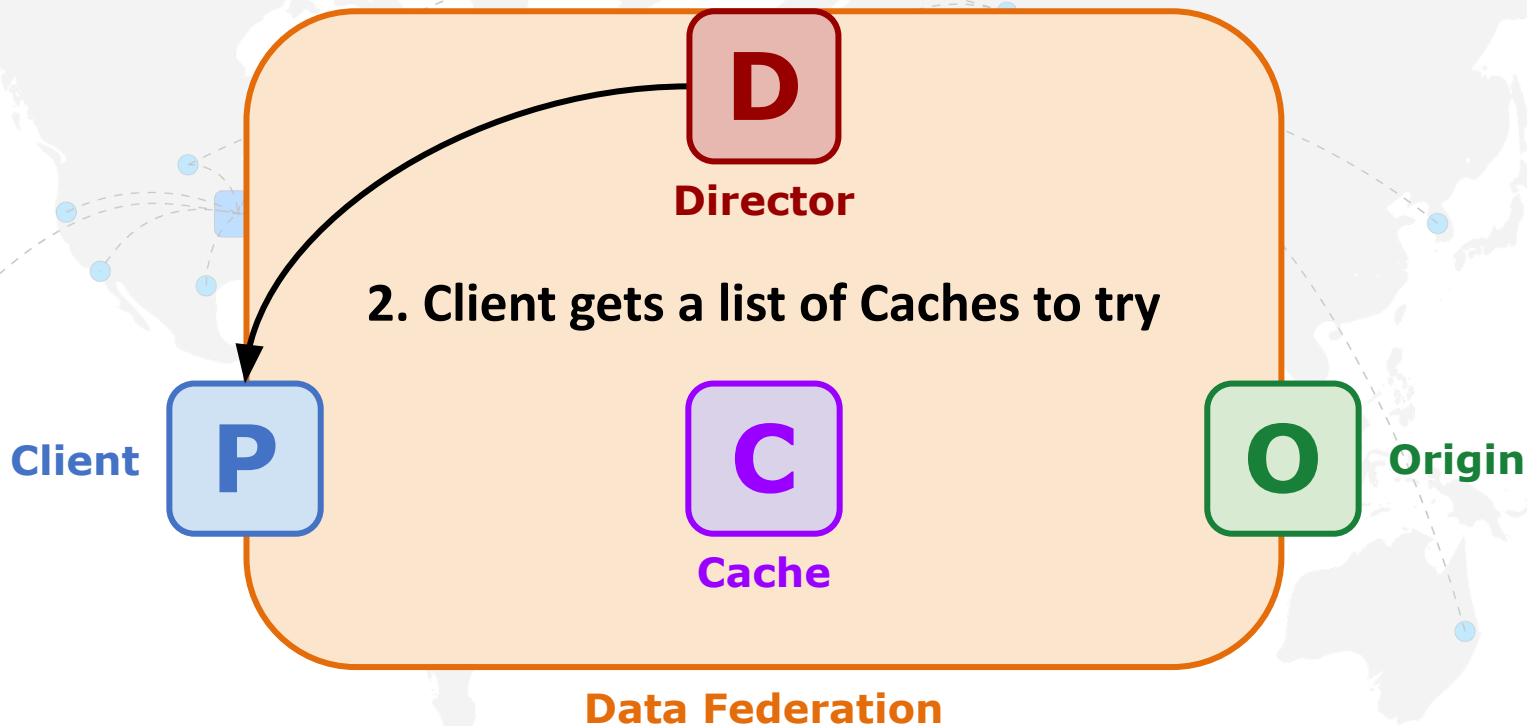


Getting an object using a Pelican URL



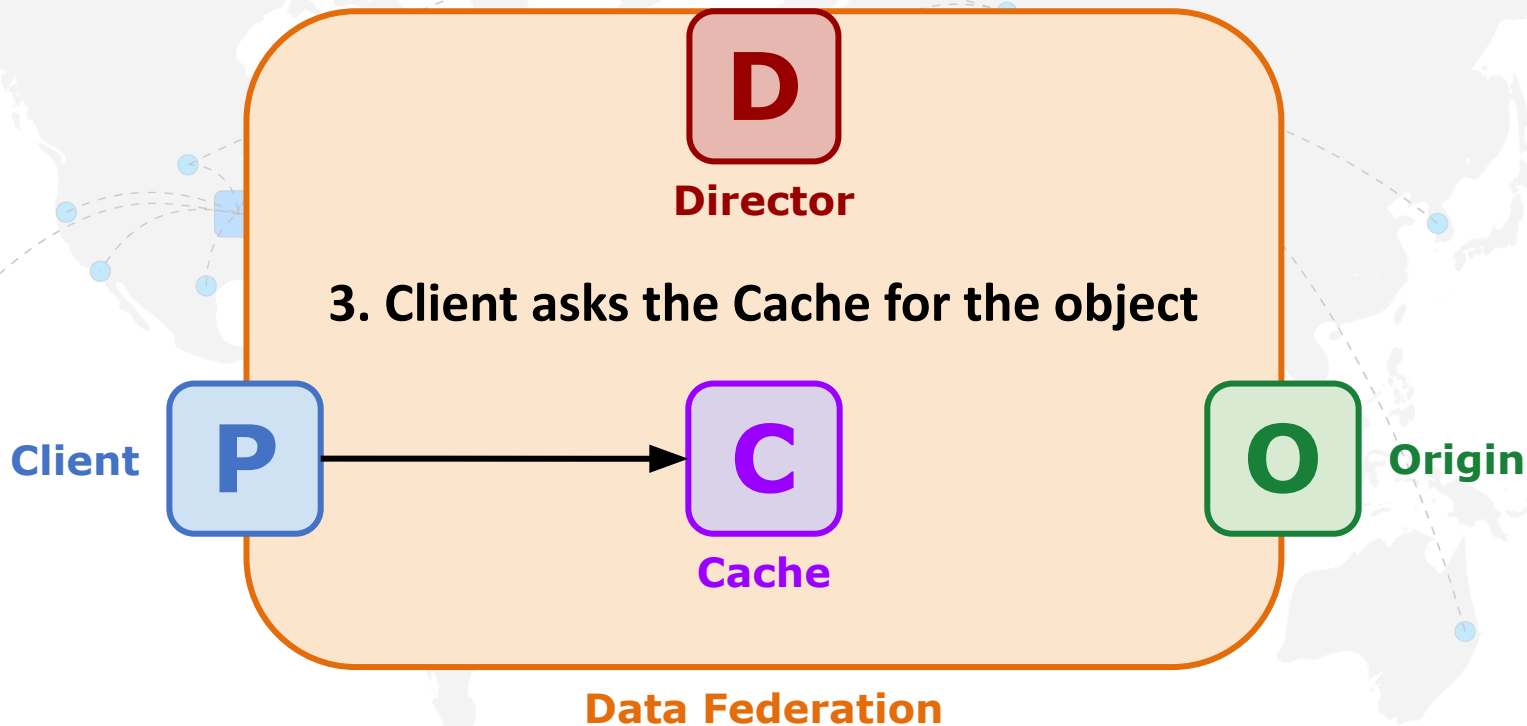


Getting an object using a Pelican URL



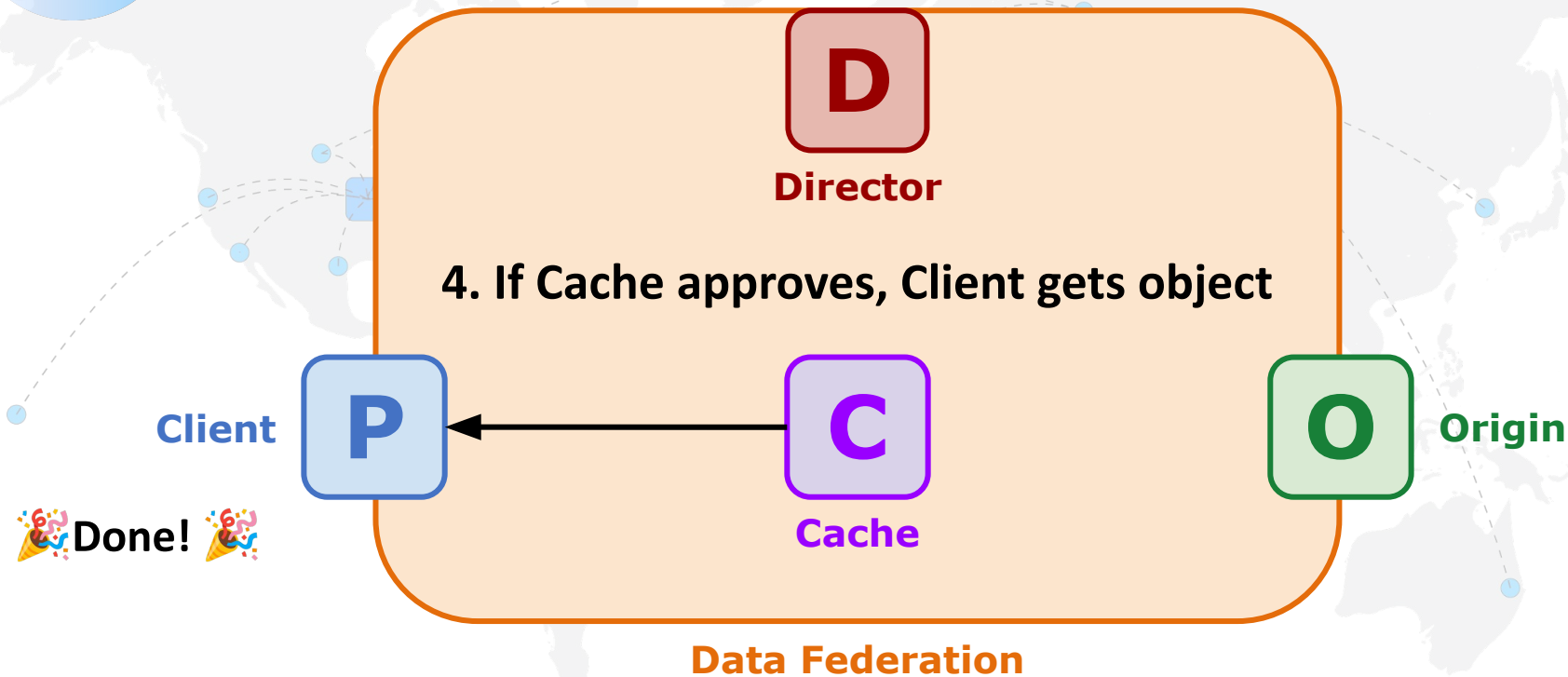


Getting an object using a Pelican URL



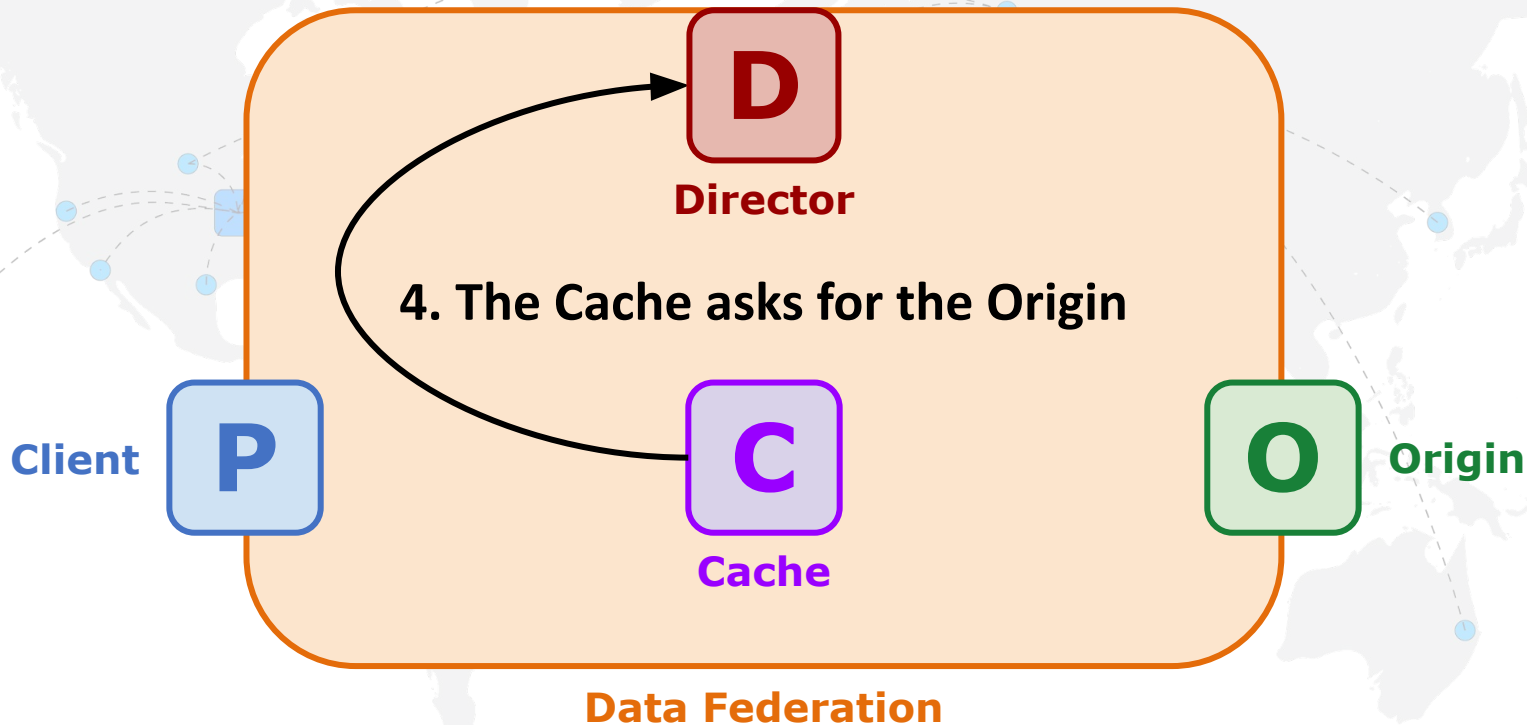


a) The Cache has the object



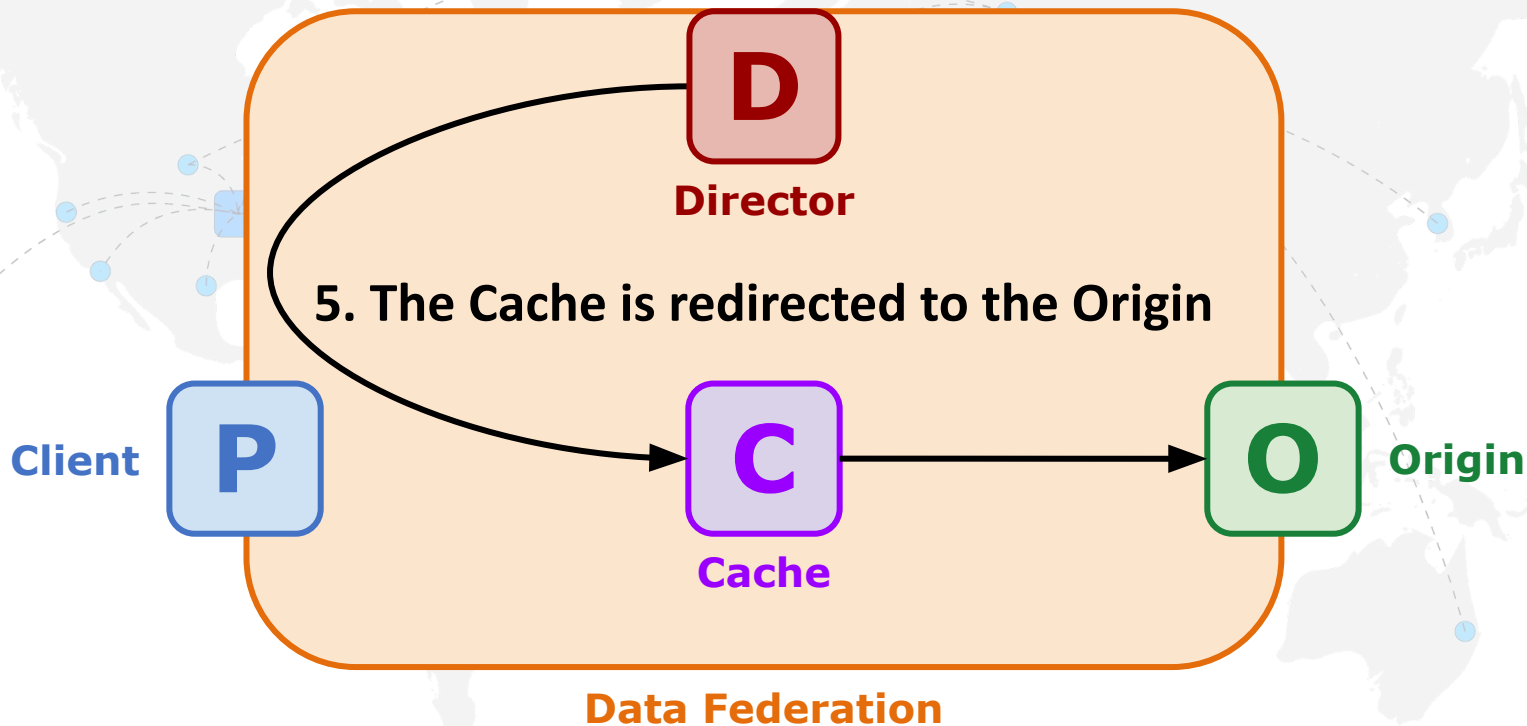


b) The Cache does not have the object



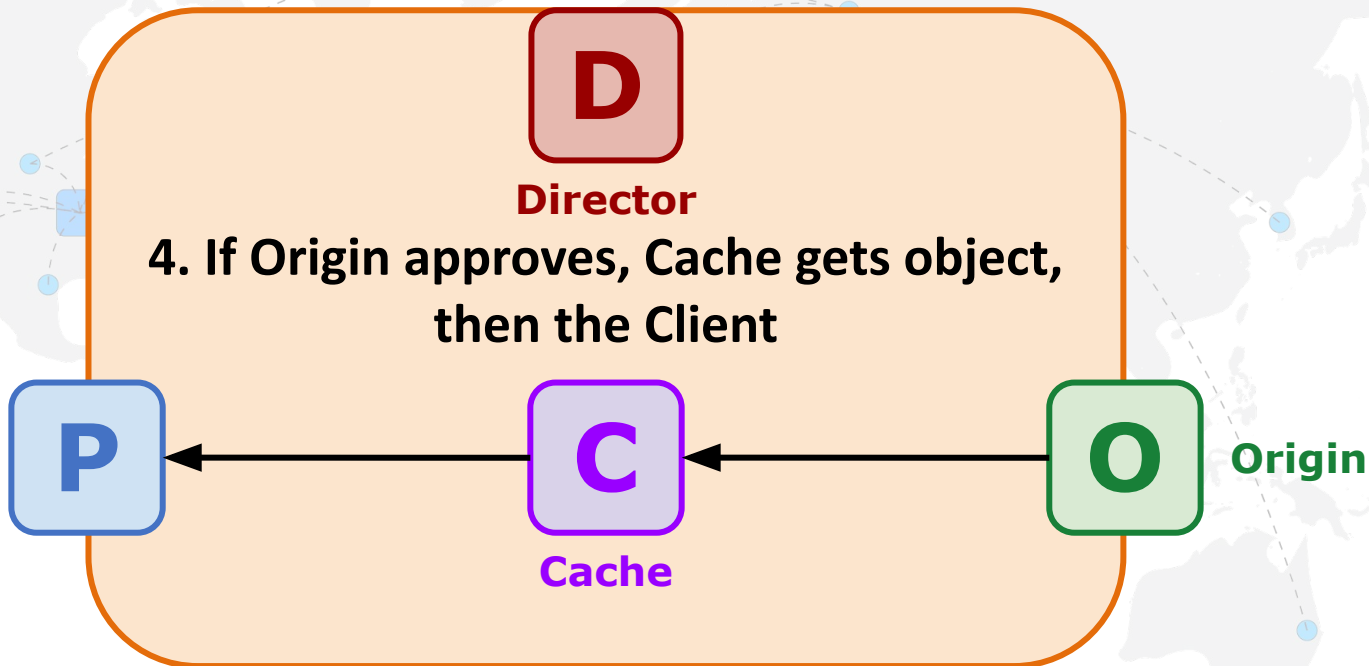


b) The Cache does not have the object





b) The Cache does not have the object



🎉 Done! 🎉

Data Federation



The importance of caching

- Pelican assumes that objects *do not change!*
- Requirements for accessing objects are respected **everywhere**
- Caches prevent the data repository from being overwhelmed

The OSDF maintains over **30 caches** across North America and Europe.
Most are high-performance servers on internet backbones.
A major advantage for connecting your data to the OSDF!



Accessing data using the Pelican Clients



Pelican Clients

Pelican provides several Clients for interacting with Pelican Federations

- [Pelican CLI](#)
- [PelicanFS](#)
- HTCondor Plugin (built-in to [HTCondor](#))



Pelican Clients

Pelican CLI

- Command Line Interface
- Available as a standalone binary

PelicanFS

- Python Interface, built on **fsspec**
- Available with pip/conda

HTCondor Plugin

- Can use Pelican/OSDF URLs in your submit files for HTCondor jobs
- Built-in to HTCondor



Basic Client actions

ls

- List the names of objects accessible via a namespace of a Pelican Federation

get

- Download a copy of an object via a Pelican Federation

put*

- Upload an object to make it available within a namespace of a Pelican Federation

*always requires authentication



Pelican CLI

Standalone binary - single file, does not require admin permissions!

docs.pelicanplatform.org/install

Linux

- [Install Pelican on Red Hat Enterprise Linux](#)
- [Install Pelican on Debian or Ubuntu](#)
- [Install Pelican on Alpine Linux](#)
- [Install Pelican as a standalone executable](#)

MacOS

- [Install Pelican on macOS](#)

Windows

- [Install Pelican on Windows](#)



Pelican CLI

OSPool Notebook setup

If using the OSPool Notebooks, make sure you are in the

`pelican-training-client`

directory!!



Pelican CLI

Once installed, available as the **pelican** command.
Uses noun-verb syntax.

Client commands use the **object** noun:

```
pelican object <request> <additional arguments>
```

(Additional arguments usually involves a Pelican URL)



Pelican CLI

Listing objects

Use this command to list the object(s) accessible via a Pelican URL:

```
pelican object ls <Pelican URL>
```

For example,

```
pelican object ls osdf:///pelicanplatform/test
```



Pelican CLI

Listing objects

Use this command to list the object(s) accessible via a Pelican URL:

```
pelican object ls <Pelican URL>
```

Remember,

pelican://osg-htc.org/_____

is exactly the same as

osdf:///_____



Pelican CLI

Listing objects

Use this command to list the object(s) accessible via a Pelican URL:

```
pelican object ls <Pelican URL>
```

For example,

```
pelican object ls osdf:///pelicanplatform/test
```



Pelican CLI

Listing objects

For more information, use the `-l/--long` flag:

```
pelican object ls --long <Pelican URL>
```

For example,

```
pelican object ls --long osdf:///pelicanplatform/test
```



Pelican CLI

Getting objects

To get an object via its Pelican URL, use the command

```
pelican object get <Pelican URL> <destination>
```

For example,

```
pelican object get  
osdf:///pelicanplatform/test/hello-world.txt ./
```




Pelican CLI

Getting objects recursively

To get all of the objects associated with a specific namespace, you can use a "query", specifically, the **?recursive** query:

```
pelican object get <Pelican URL>?recursive <destination>
```

For example,

```
pelican object get  
osdf:///pelicanplatform/test?recursive ./
```



Pelican CLI

Putting objects

To put an object via its Pelican URL, use the command

```
pelican object put <local source> <Pelican URL>
```

This action **always** requires authentication. The client will prompt you to provide the necessary credentials, if possible.

To approve credentials for this operation, please navigate to the following URL and approve the request:

```
https://tutorial-origin.svc.osg-htc.org/api/v0.0/issuer/device?user\_code=EXA\_3DB\_285
```



Pelican CLI

Putting objects

This exercise will only work **in the OSPool Notebook**.

First, open a "Terminal" in the Jupyter interface and run this command:

```
pelican credentials reset-password
```

Leave the password empty and press the Enter key when prompted.



Pelican CLI

Putting objects

Switch back to the `pelican-CLI` notebook tab. Run the provided command to define a ~random value for `my_unique_object_name`.

For the put command to work, the object name must be unique!



Pelican CLI

Putting objects

Now run the put command:

```
pelican object put hello-world.txt  
osdf:///osdf-tutorial/protected/${my_unique_object_name}
```

When prompted, open your unique URL in the browser and login **the same way you logged into the OSPool Notebook.**



Pelican CLI

Putting objects

Confirm the object was uploaded as intended, using the `ls` or `get` commands.

Can you come up with the necessary commands without looking?

You should see that the object was uploaded with your unique name!



Pelican CLI

Getting/putting LOTS of objects

The CLI has a "sync" verb for transferring lots of objects.

To get lots of objects:

```
pelican object sync <Pelican URL> <destination>
```

To put lots of objects:

```
pelican object sync <local source> <Pelican URL>
```

If the transfer is interrupted, rerunning the command will automatically skip already-transferred objects.



A peak behind the curtain...

Using '--debug'

Verbose information about the Pelican Client can be reported using the **--debug** flag.

For example,

```
pelican object get --debug  
osdf:///pelicanplatform/test/hello-world.txt
```




PelicanFS

For a Python-based Client, use Pelican's **pelicanfs** Python package

Install it with **pip** like other Python packages:

```
python3 -m pip install pelicanfs
```

This provides a Pythonic interface to interact with a Pelican Federation, usable in scripts and the Python console



PelicanFS

Import and setup

First, launch the `python3` terminal console by entering
`python3`



PelicanFS

Import and setup

The `pelicanfs` client provides the `PelicanFileSystem` class, based on the `fsspec` package.

Next, import the necessary class:

```
from pelicanfs.core import PelicanFileSystem
```



PelicanFS

Import and setup

The `pelicanfs` client provides the `PelicanFileSystem` class, based on the `fsspec` package.

Now instantiate an instance of the object, defining the Discovery URL when you do so:

```
pelfs = PelicanFileSystem('pelican://osg-htc.org')
```



PelicanFS

Client Methods

Now instantiate an instance of the object, defining the Discovery URL when you do so:

```
pelfs = PelicanFileSystem('pelican://osg-htc.org')
```

The methods of this object provide are the mechanisms of the Client and take the form

```
pelfs.<method>(<arguments>)
```

where **<arguments>** usually contains the rest of the Pelican URL



PelicanFS

Listing objects

List objects by providing the namespace to the `ls` method:

```
pelfs.ls('/namespace')
```

The result will be a list of dictionaries. For example,

```
list_results = pelfs.ls('/pelicanplatform/test')
```

To see just the names of the objects, run

```
[result['name'] for result in list_results]
```



PelicanFS

Getting objects

Use the **get** method to get an object:

```
pelfs.get('/namespace/object_name', 'destination')
```

For example,

```
pelfs.get('/pelicanplatform/test/hello-world.txt',  
          'hello-world.txt')
```



PelicanFS

Getting objects

Then use the file the typical Python way:

```
with open('hello-world.txt', 'r') as f:  
    my_file = f.read()  
  
print(my_file)
```




PelicanFS

Getting objects the **fsspec** way

Instead of manually copying the object to the local file system, let **fsspec** do it for you:

```
with pelfs.open('/pelicanplatform/test/hello-world.txt',  
                'r') as f:  
    direct_read = f.read()  
  
print(direct_read)
```



PelicanFS

Automatically (!) get objects

Lots of Python packages automatically use `fsspec` behind the scenes for data transfers. The same is true for `pelicanfs`!!

To demonstrate this, exit the Python console:

```
exit()
```



PelicanFS

Automatically (!) get objects

Take a look at the included script:

```
cat autoload.py
```

Note that **pelicanfs** is not referenced anywhere in the script!



PelicanFS

Automatically (!) get objects

Now execute the script:

```
python3 autoload.py
```

After a minute or two, you'll see the dataset!

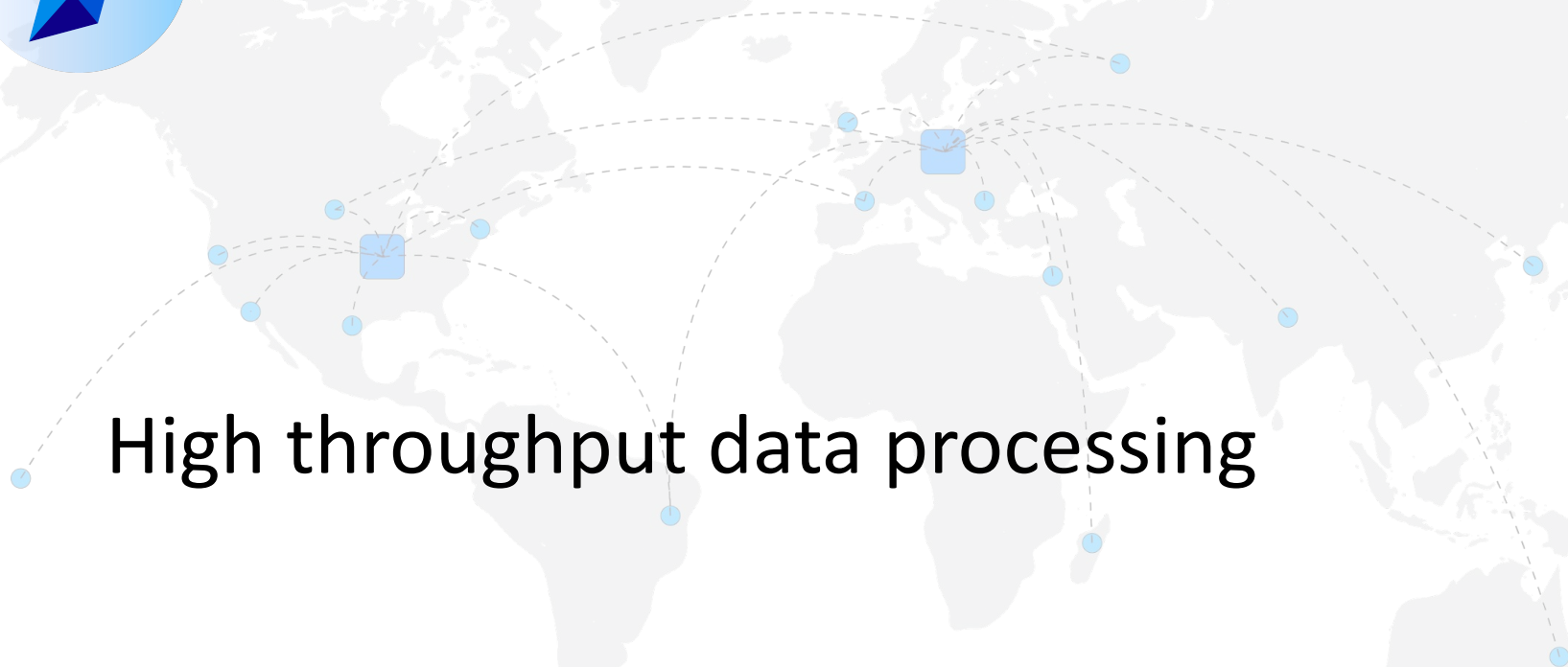
Behind the scenes, **pandas** knows to use the **pelicanfs** package to download data with **osdf://** and **pelican://** URLs



PelicanFS

Notes about **pelicanfs**:

- Not all features of **fsspec** are available (since the Pelican Client is not a proper filesystem..)
- Still in development!



High throughput data processing



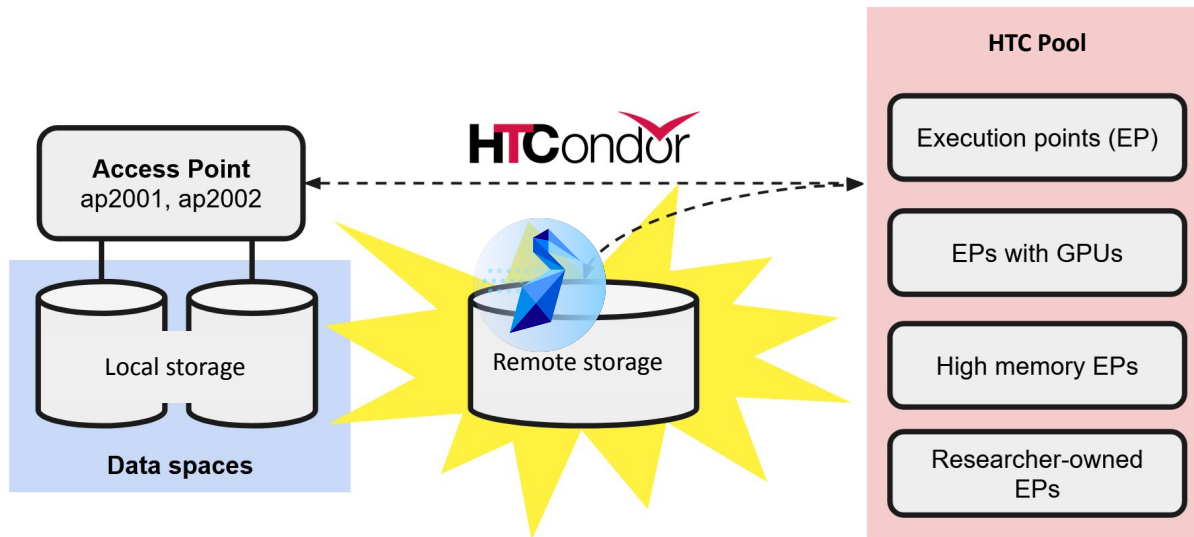
Motivation

So far, we have just used Pelican to upload, download and stream.

But what if you have a dataset you want to compute on (without staging the whole dataset first)?



We can do this!!



If you have data in a Pelican data federation, you can use it in jobs.



HTCondor and Pelican

Pelican has tight integration with HTCondor

Among other things, **can use Pelican URLs in your submit file!**

- Normal declaration, but using `pelican://` or `osdf://` file transfer protocol
- HTCondor manages the transfer
- Errors are automatically retried, or turned into holds



Back to notebooks...

<https://osg-htc.org/notebooks>



Data & Compute at Scale

To demonstrate the power of Pelican's HTCondor Plugin, let's work on a small exercise using [weather data from NOAA](#)

The screenshot shows the NOAA National Centers for Environmental Information website. The header includes the NOAA logo and the text "National Centers for Environmental Information" and "NATIONAL OCEANIC AND ATMOSPHERIC ADMINISTRATION". A navigation bar contains links for Home, About, News, Access, Archive, and Contact, along with a search bar. The breadcrumb trail reads "Home > Geoportal > Dataset Overview". The main heading is "Global Historical Climatology Network - Daily (GHCN-Daily), Version 3". Below this is a section titled "Daily Summaries" which includes a small map of the world labeled "Map of GHCNd stations" and a paragraph of text describing the dataset. To the right of the text are three buttons: "Dataset Citation", "Dataset Identifiers", and "ISO 19115-2 Metadata".

NOAA National Centers for Environmental Information
NATIONAL OCEANIC AND ATMOSPHERIC ADMINISTRATION

Home About News Access Archive Contact Search

Home > Geoportal > Dataset Overview

Global Historical Climatology Network - Daily (GHCN-Daily), Version 3

Daily Summaries


Map of GHCNd stations

The Global Historical Climatology Network - Daily (GHCN-Daily/GHCNd) dataset integrates daily climate observations from approximately 30 different data sources. Version 3 was released in September 2012 with the addition of data from two additional station networks. Changes to the processing system associated with the version 3 release also allowed for updates to occur 7 days a week rather than only on most weekdays. Version 3 contains station-based measurements from well over 90,000 land-based stations worldwide, about two

[Dataset Citation](#)
[Dataset Identifiers](#)
[ISO 19115-2 Metadata](#)



The climate dataset

The GHCN dataset from NOAA is available via the OSDF!

- Part of Amazon's Open Data repository (existing namespace at [/aws-opendata](#))
- Connected via US East 1 ([us-east-1](#))
- S3 domain name is [noaa-ghcn-pds](#)

Altogether, the data should be accessible via

[osdf:///aws-opendata/us-east-1/noaa-ghcn-pds](#)



The climate dataset

Structure looks like this (ignoring lots of other files..)

```
osdf:///aws-opendata/us-east-1/noaa-ghcn-pds/  
ghcnd-stations.txt  
csv/  
    by_station/  
        <STATION_ID_1>.csv  
        <STATION_ID_2>.csv
```



Explore the dataset

In Jupyter, move into the `htcondor-plugin` directory.

Then get the stations list using one of the Clients.

Recommend copying the `pelican object get` command from the Github README (under the "Exploring the data" subheading)



Explore the data

Download a csv file for a station using Pelican.

Example command at the end of the "Exploring the data" subheading.

Should see climate data for a specific weather station.



Explore the analysis

Then run the `example.py` script with the station ID as an argument.

For example,

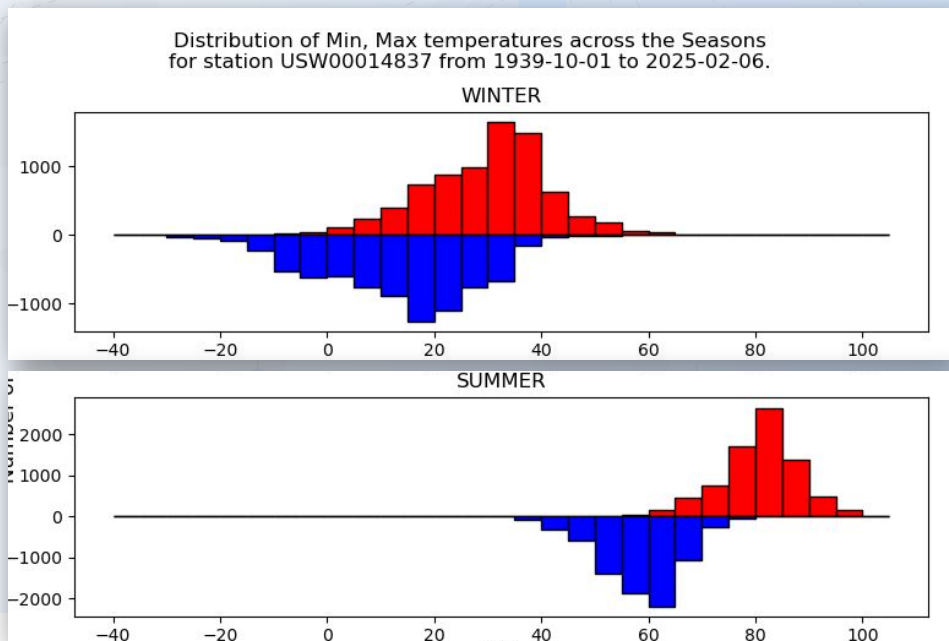
```
./example.py USW00014837
```

This will create a `.png` file!



The climate "analysis"

Winter is colder than summer! (at least in Wisconsin)





Scaling out

Suppose you want to analyze ALL the stations

- Dataset contains ~130,000 stations!!

Suppose you also want to do a better analysis, but it takes 1 hour per station

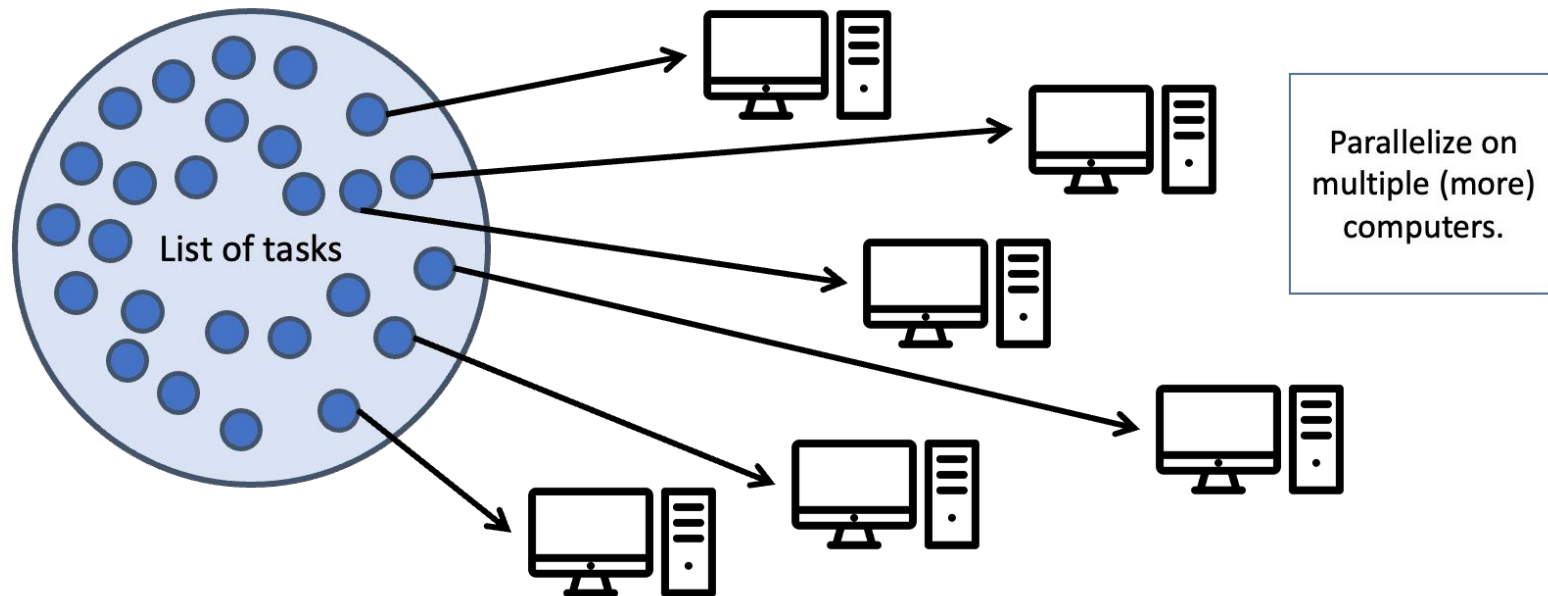
- Running all stations in serial (one after the other) will take ...

$$130,000 \text{ stations} \times \frac{1 \text{ hour}}{1 \text{ station}} = \mathbf{15 \text{ years}}$$

💀 💀 💀

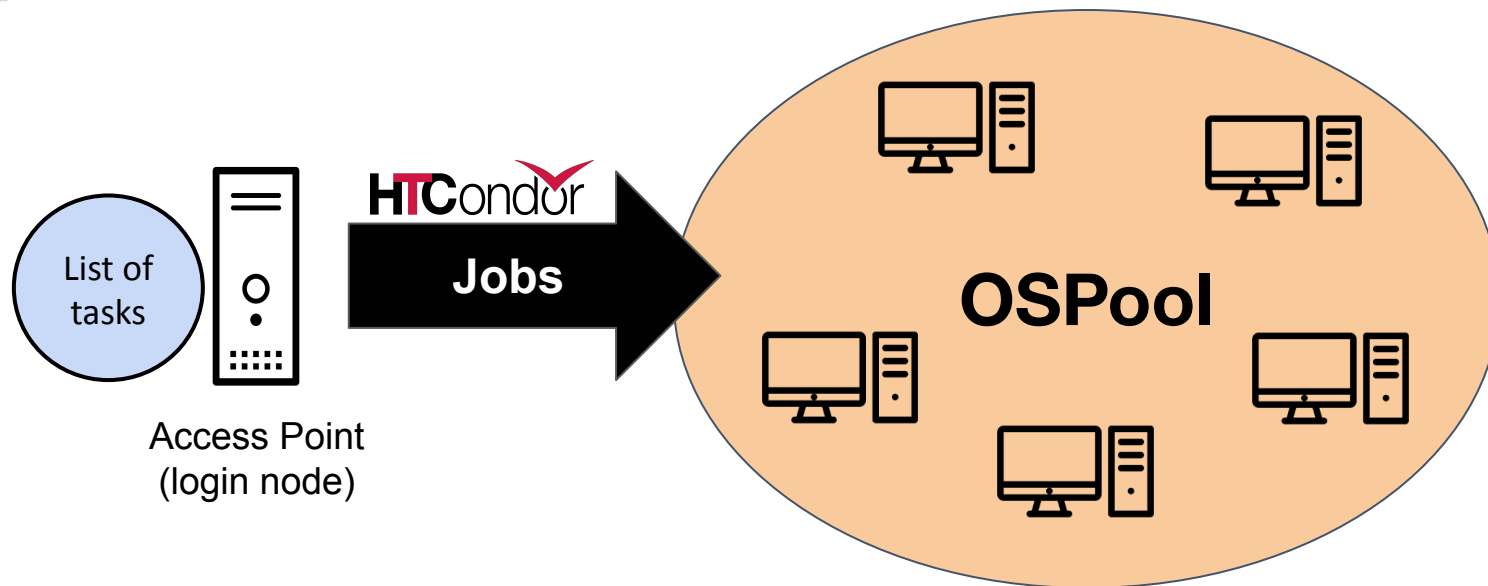


HTC Approach





OSPool





OSPool Size and Scale

Open Science Pool | OSG

osg-htc.org/services/ospool/

Services Campuses Community Contact About Docs

OSPool Overview Projects Institutions Documentation

**OSPool: Serving Open Science
throughput computing**

On July 19
1M jobs completed
Placed by 58 researchers
Triggering 11M file transfers
Consuming 723K core hours

[View active OSPool Projects](#) [Contact Us!](#)





OSPool Size and Scale

```
ckoch — christina.koch@ospool-ap2140:~ — ssh christina.koch@ap40.uw.osg-htc.org — 110x25
```

wn-lb-13.gina.surf.nl				x64/CentOS9	6	4	0
wn-lb-14.gina.surf.nl				x64/CentOS9	5	4	0
wn-lb-15.gina.surf.nl				x64/CentOS9	9	4	0
wn-lb-16.gina.surf.nl				x64/CentOS9	2	4	0
wn-lb-17.gina.surf.nl				x64/DEBIAN12	9	4	0
wsu-lc01.osris.org				x64/CentOS9	45	1	0
wsu-lg01.osris.org				x64/CentOS9	1505	1	0
wsu-lm01.osris.org				x64/CentOS9	78	8	0
wsu-lm02.osris.org				x64/AlmaLinux9	0	8	0

	Total	Owner	Claimed	Unclaimed	Matched	Preempting	Drain	Backfill	BkIdle
aarch64/CentOS9	1	0	0	1	0	0	0	0	0
aarch64/Rocky9	1	0	0	1	0	0	0	0	0
x64/AlmaLinux9	77	0	62	15	0	0	0	0	0
x64/CentOS7	243	0	172	71	0	0	0	0	0
x64/CentOS8	3745	0	3434	303	0	8	0	0	0
x64/CentOS9	33801	0	31140	2604	0	57	0	0	0
x64/DEBIAN12	1366	0	1246	120	0	0	0	0	0
x64/RedHat9	1	0	0	1	0	0	0	0	0
x64/Rocky9	1	0	0	1	0	0	0	0	0
x64/UBUNTU22	1600	0	1472	126	0	2	0	0	0
Total	40836	0	37526	3243	0	67	0	0	0

```
[christina.koch@ap40 ~]$
```



Scaling out with the OSPool

High Throughput Computing (and HTCondor) to the rescue!

- On a system like the OSPool*, can easily run 1000s of jobs at a time

$$130,000 \text{ stations} \times \frac{1 \text{ hour}}{1,000 \text{ stations}} = 130 \text{ hours} \approx \mathbf{5 \text{ days}}$$



Explore HTCondor

In the `htcondor-plugin` directory, generate a list of station ids and then take a look at the example submit file:

```
./generate_list.sh  
cat partial-example.sub
```

Does each item in the submit file make sense? Where might we integrate the Pelican URL for our data files?



Scaling out with the OSPool

High Throughput Computing (and HTCondor) to the rescue!

- On a system like the OSPool, can easily run 1000s of jobs at a time

$$130,000 \text{ stations} \times \frac{1 \text{ hour}}{1,000 \text{ stations}} = 130 \text{ hours} \approx \mathbf{5 \text{ days}}$$

What about the data movement?



Scaling out with HTCondor and Pelican

Pelican (and the OSDF) to the rescue!

- You don't need to manually stage the dataset - just give HTCondor the correct Pelican URLs

For example,

```
OSDF_PREFIX =  
osdf:///aws-opendata/us-east-1/noaa-ghcn-pds/csv/by_station/  
transfer_input_files = $(OSDF_PREFIX)/$(STATION_ID).csv  
queue STATION_ID from station_list.txt
```



Pelican Plugin

Transferring as input

In the submit file, inputs to be transferred are declared using `transfer_input_files`. Just put the Pelican URL in that list for the object you want to transfer as input.

For example,

```
transfer_input_files =  
osdf:///pelicanplatform/test/hello-world.txt
```



Submit a list of climate analysis jobs

In the `htcondor-plugin` directory, take a look at the completed example submit file:

```
cat example.sub
```



Submit a list of climate analysis jobs

Submit the list of jobs with

```
condor_submit example.sub
```

Monitor the progress of the jobs with

```
condor_watch_q
```

If you really wanted to do a complex analysis of all 130,000 stations,
please don't use the Guest OSPool Notebook to do so!
Instead, request a full OSPool account at portal.osg-htc.org/application



Pelican Plugin

Transferring as output

You can also transfer output back into an external object store via a Pelican origin, using HTCondor's `transfer_output_remaps` or `output_destination` options with a Pelican URL.

1)

```
transfer_output_remaps =  
"result.txt=osdf:///pelicanplatform/test/result.txt"
```

2)

```
output_destination = osdf:///pelicanplatform/test
```



Concluding Remarks



Next Steps

More information on Pelican:

pelicanplatform.org

docs.pelicanplatform.org

More information about the OSDF:

osg-htc.org/services/osdf

osg-htc.org/services/osdf/data

Request an OSPool account:

<https://portal.osg-htc.org/application>



Trainings and Getting Help

For OSDF/OSPool support, email: support@osg-htc.org

For Pelican support, email: help@pelicanplatform.org



Authorization and Sharing in the OSDF

https://drive.google.com/file/d/1IOMXciVY_aa2V9UgGiT2RTUvItcFiDiS/view?usp=sharing

A light gray world map serves as the background. Overlaid on the map are several blue circular data points and two blue square nodes. Dashed gray lines connect these points and nodes, forming a network that spans across North America, Europe, and Asia. The word "Questions?" is written in large black font over the map.

Questions?



Acknowledgements

This material is based upon work supported by the National Science Foundation under Cooperative Agreements OAC-2331480. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.